One Frozen DINOv3, Wide LoRA and a Tiny-Object Cascade: Multi-Task Learning on SAR Images under a 40 M Budget

Rayner Sim Zhi Heng Chen Junkai Pei Jinling Le Ba Luat
CS701 Deep Learning and Vision, Team 7, Singapore Management University

Abstract

One model must classify a synthetic aperture radar (SAR) image into nine scene classes and detect all of its objects, half of which are smaller than one 16×16 *ViT patch, with at most 130 M parameters, 40 M of them trainable, and one forward pass at* $\leq$ $512\,px$. *We keep a DINOv3 ViT-B/16 frozen and spend the trainable budget on rank-80 LoRA and a three-stage Cascade R-CNN rebuilt for tiny boxes: 6 px anchors matched by* $\max(\mathrm{IoU}, \mathrm{NWD})$, *IoU thresholds that relax with box size, a pixel-level CNN prior, and a fix for a* $-0.3\,px$ *annotation bias that ordinary flips double. Zero-cost steps on one set of weights (checkpoint averaging, image–box score fusion, soft-NMS and snapping to an annotation lattice) add a further* $\approx 3.6\,\Delta m$. *The entry scores* Δm *56.49 on the validation board and 61.25 on the test board (macro-F1 96.96, mAP 55.74), third of nine teams at the 8 Nov board snapshot. A parallel CNN line (YOLO11, ConvNeXt Cascade R-CNN) plateaued at* Δm *40.35, and three cases show that gains seen in short runs or on one local holdout shrink by* 5–$10\times$ *or vanish on the board.*

1. Introduction

SAR sensors image the ground day and night through cloud, which makes them valuable for monitoring ports, airports and energy infrastructure. A SAR image is a map of microwave backscatter, not a photograph: it is single-channel, covered in multiplicative speckle, and an object often appears as a few bright scatterers next to a dark shadow. Generic vision models are therefore not obviously transferable, and SAR-specific pretraining is excluded by the course rules.

The task couples two outputs from one image: a scene label (macro-F1) and boxes for every object of that class (COCO mAP@[.5:.95]). Teams are ranked by

$$\Delta m = 50\left[\frac{F1-F1_{\mathrm{ref}}}{F1_{\mathrm{ref}}} + \frac{\mathrm{mAP}-\mathrm{mAP}_{\mathrm{ref}}}{\mathrm{mAP}_{\mathrm{ref}}}\right], \qquad (1)$$

with a ViT-B + LoRA reference of 93.31 / 26.13 on validation and 93.99 / 25.41 on test. Since $F1_{\mathrm{ref}}$ is already high, one mAP point is worth $1.91\,\Delta m$ and one F1 point only 0.54: **the score is earned by detection**, which became our first design decision.

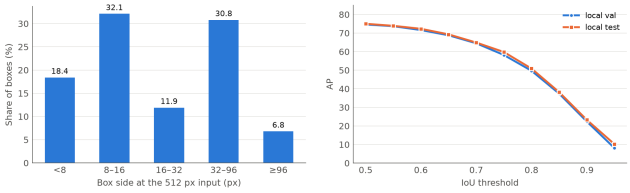


Figure 1. Left: share of boxes by side $\sqrt{wh}$ at the 512 px input; half are below the 16 px patch size. Right: AP against IoU threshold on our two local holdouts; the loss concentrates at strict IoU.

Three properties of the data then shaped the model (Fig. 1). (i) **Boxes are tiny**: 18.4% of the 32,533 boxes have side < 8 px and 50.5% are below 16 px, *i.e.* below one ViT token. (ii) **Localisation, not recognition, is the bottleneck**: our AP falls from $\approx$75 at IoU 0.5 to $\approx$22 at 0.9. (iii) **Annotations carry structure**: a systematic sub-pixel offset, an integer coordinate lattice for some classes, about 10% near-duplicate images, and two classes (bridge, playground) whose labels appear only partly recoverable from SAR.

Our objectives were therefore to (1) adapt a strong generic backbone within 40 M trainable parameters without destroying its features, (2) make a ViT detector work below its patch size, (3) extract every zero-cost gain from a single forward pass, and (4) build an evaluation protocol that tells real gains from noise. Our contributions are a tiny-object Cascade on a frozen DINOv3, a set of data-driven processing steps, and a candid account of what did not transfer from local runs to the board.

2. Related work

Multi-task learning. Hard parameter sharing with task heads is the simplest MTL design; loss weighting [8] and task-specific attention [12] address task interference, and Δm is the standard summary metric [17]. Here the two tasks are strongly aligned (every box shares the image

class), so interference is small; we found it more useful to let each task inform the other at inference (score fusion, Sec. 3.1) than to tune loss weights.

Parameter-efficient adaptation. LoRA [6] adds low-rank updates to frozen weights, and MTLoRA [1] splits them into shared and task-specific parts. LoRA keeps pre-trained features intact and is cheap to store, but its capacity is bounded by the rank; full fine-tuning of a few blocks is the main alternative. We compare both under the same budget (Sec. 3.3).

Backbones. Self-supervised ViTs such as DINOv2/v3 [13, 16] give strong dense features, but a stride-16 token map [5] has no pixel detail for 8 px objects. CNNs such as ConvNeXt [11] keep stride-4 features and fit the budget when fully trained, but their supervised ImageNet features transferred worse to SAR in our tests.

Detectors. Two-stage detectors [15] with FPN [10] are accurate but tied to anchors; Cascade R-CNN [3] refines boxes in stages with rising IoU thresholds, which directly targets strict-IoU AP. One-stage YOLO detectors [14] and DETR-style real-time detectors [20] are fast and simple to train, but in our runs they were weaker at strict IoU on tiny boxes. ViTDet [9] builds a pyramid from a single ViT map, and ViT-Adapter [4] injects a CNN spatial prior; we use both ideas in reduced form.

Tiny objects. IoU is unstable for small boxes: a 2 px offset already drives the IoU of an 8 px box towards zero. NWD [18] models boxes as Gaussians and replaces IoU in assignment. Soft-NMS [2] keeps overlapping true positives, and weight averaging along a trajectory [7, 19] improves a single model at no inference cost.

3. Method

Figure 2 shows the final model; each part below states what we changed and why.

Backbone and adaptation. We use timm's DINOv3 ViT-B/16 (LVD-1689M) and sum the RGB patch kernel into one gray channel, so the pretrained weights are reused exactly on SAR input. The backbone stays frozen. LoRA of rank 80 (α=160) is added to qkv, proj, fc1 and fc2 in all 12 blocks, and the LayerNorms train. Images of 256 or 224 px are stretched to 512, which the rules allow (one pass, at most 512^2, no tiling).

Pyramid and spatial prior. A ViTDet-style pyramid (strides 4–64, 256 channels) is built from the last token map. Because a stride-16 map has no detail for sub-patch objects, a randomly initialised 64-channel CNN (0.21 M parameters) reads the raw image and is added to the stride-4 and stride-8 levels through zero-initialised 1×1 convolutions, so training starts from the unmodified pyramid.

Tiny-object RPN. Anchors range over 6–181 px and sit at cell centres. Positives are assigned by $\max(\mathrm{IoU}, \mathrm{NWD})$ with C=8, so 8 px objects that IoU would reject still receive anchors.

Size-aware Cascade. The standard 0.5/0.6/0.7 stage thresholds leave tiny objects without positives in later stages. We use thresholds that depend on GT side: .40/.45/.50 below 8 px, .45/.50/.55 for 8–16 px, .50/.55/.65 for 16–32 px, and .5/.6/.7 above. Heads are class-agnostic, 1536 wide, with aligned RoIAlign.

Flip-bias correction. Boxes carry a systematic offset of about -0.3 px. A horizontal flip mirrors it, so flipped targets disagree with unflipped ones by twice the bias; our flip moves flipped boxes back by -0.3 px.

Training. Flips only (rotations hurt; see Sec. 4.5); AdamW, LR 10^{-4} for heads and 3×10^{-4} for LoRA, weight decay 0.05, batch 16, a 200-epoch cosine schedule, and all 9,392 labelled images for the submitted model. Training took 13 h on two RTX 6000 Ada GPUs.

3.1. Single-pass post-processing

Checkpoint averaging. The weights at epochs 100, 110, ..., 180 of one run are averaged into one model, which still costs one forward pass. **Box score.** With p the stage-averaged box class probabilities and p_{img} the image classifier, $s_c = (1 - p_{\text{bg}})\, p_{\text{img}}(c)$, without RPN objectness. This class-agnostic fusion lets the image prior suppress other-class false positives without deleting boxes in misclassified images. **Boxes** are the mean of the three stages, shifted by -0.2 px, with per-class Gaussian soft-NMS (σ=0.5), and at least 100 boxes are kept per image. **Image label** is $\arg\max_c [\log p_{\text{img}}(c) + 2 \log \max_i p_i(c)]$, so confident detections can correct the classifier. **Lattice snapping.** All training boxes of aircraft, car and every 256 px image have integer coordinates in original pixels, and about 40% of wind-turbine images share one fractional offset. We round predicted corners onto that lattice; for wind turbines a per-image circular-mean offset is applied when its concentration $R \geq 0.5$. Scores and ranking are unchanged.

3.2. A parallel CNN detector line

To test whether a convolutional design with native stride-4 features suits 8 px targets better, one of us developed a CNN line on the same split and protocol. It started from YOLO11-L (COCO-pretrained) and moved to a torchvision Faster/Cascade R-CNN on a ConvNeXt-T/S backbone (ImageNet-22k) with an FPN over P2–P6. The ConvNeXt-S version keeps the stem and early stages trainable and freezes part of stage 2, giving 62.6 M total and 39.5 M trainable parameters. RPN and RoI computations had to stay in FP32, because bf16 autocast snapped boxes to a 2 px grid. The line then added the same kinds of change as the main model: soft-NMS, image–box score fusion, the box shift, and later 6 px anchors, an NWD term in the RPN and size-aware IoU.

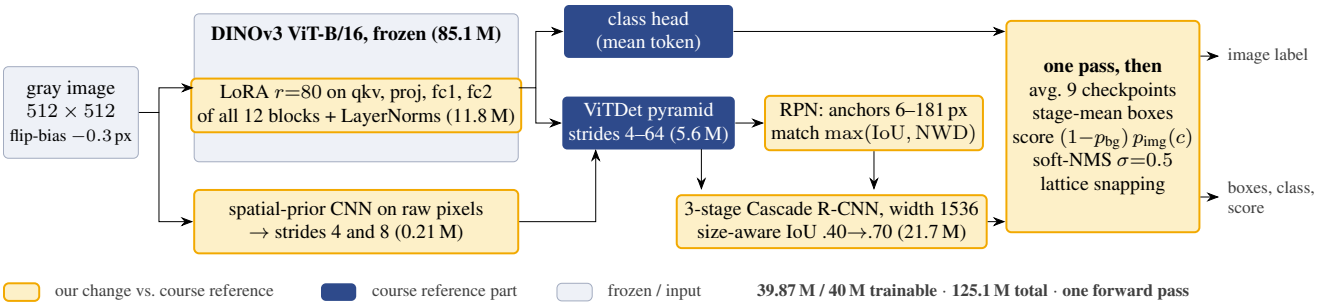


Figure 2. Final model. Amber blocks are our changes to the course reference (ViT-B + LoRA + Faster R-CNN). Each image goes through the network once; everything in the right-hand box uses only that forward pass.

Entry	Main change	F1	mAP	Δm
V2	DINOv3 + LoRA + Cascade, 120 ep	95.06	45.62	38.23
F2	+ size-aware IoU, flip bias, all data	95.46	48.97	44.84
SC2	LoRA r80, head 1536, 150 ep	95.35	52.66	51.86
BP7	+ spatial prior, cosine 160	95.35	53.15	52.78
cos200	cosine 200, e160	95.02	53.56	53.41
	+ post-proc. (Sec. 3.1)	95.09	54.29	54.83
	+ avg. 9 ckpts, agn. fusion	95.27	55.10	**56.49**
Test entry (intended recipe + snapping)		96.96	55.74	**61.25**

Table 1. Official scores (validation board; last row test board, with its own reference). Steps change several things at once; Tabs. 2 and 3 attribute them.

Variant (one part removed)	Train. M	mAP	$\Delta\Delta m$
Final recipe	39.87	45.76±0.09	–
no Cascade (Faster R-CNN)	32.16	42.23±0.34	−7.09±0.46
LoRA r32, head 1024	24.02	43.87±0.35	−3.63±0.49
no size-aware IoU	39.87	44.60±0.53	−2.31±1.11
reference anchors	39.86	44.87±0.33	−1.86±0.80
no flip-bias fix	39.87	44.88±0.43	−1.54±0.84
no spatial prior	39.66	45.47±0.35	−0.71±0.40
LayerNorms frozen	39.83	45.75±0.15	−0.13±0.29
no NWD matching	39.87	46.10±0.51	+0.60±1.20

Table 2. Leave-one-out on *vallike* (36-epoch cosine, full post-processing; mean ± std over 3 seeds, paired by seed). Short runs rank parts but understate parts that pay late.

3.3. Spending the budget on full blocks

LoRA of rank 80 everywhere is one way to spend 40 M parameters. The alternative trains the *last two ViT blocks fully* (14.2 M), uses LoRA of rank 48 on the other ten blocks and a 1024-wide head (39.15 M trainable). We screened this and two spatial-prior variants on the team code, then trained the winner at full length.

4. Experiments and analysis

Protocol. Screens use a class-stratified split of the training set (7,512 / 940 / 940). Ablations for this report use a second split, *vallike*: 992 held-out images sampled to match the predicted class and image-size mix of the official validation set, with the other 8,400 images for training. Official validation and test images were only ever predicted. Repeating one 36-epoch configuration with 37 seeds gave a standard deviation of ± 0.70 local Δm, so differences between single runs below $\approx 1\ \Delta m$ are noise. Post-processing comparisons are paired on the same weights, so 0.1–0.3 Δm differences can be real there.

4.1. Main result and progression

Table 1 traces the official score. The largest single jump (F2→SC2, $+7.0\ \Delta m$) came from filling the budget with a wider LoRA and head together with a longer schedule; slower cosines helped twice ($+1.3$, $+0.6$), and the last 3.1 Δm on validation needed no training at all. We finished Phase 1 third of nine and were third of nine on the test board at the 8 Nov snapshot (the board closed later that day). Our test macro-F1 (96.96) is above both leading teams (96.11, 96.27), whereas their mAP is 5.5 points higher (61.2 *vs.* 55.7) while their AP50 is only 2.6 higher, so the remaining gap is at strict IoU.

4.2. What each part adds

Table 2 removes one part at a time. The cascade is the most valuable part ($-7.1\ \Delta m$ without it), confirming that the bottleneck is box quality. Capacity is next: shrinking LoRA and the head to 24 M costs 3.6 Δm. All three tiny-object changes help (size-aware IoU, small anchors, flip bias, together $\approx 5.7\ \Delta m$). NWD matching shows no stable benefit once the other tiny-object parts are present: seed 0 declines and seeds 1–2 improve. Earlier single-seed screens on the 7,512 split ranked the same parts in the same order: DINOv3 instead of ImageNet-21k ViT gave $+8.1$ local Δm, LoRA on all linear layers $+18.1$, and the cascade $+21.0$. Table 3 shows the zero-training steps. Checkpoint averaging is the largest ($+1.38\ \Delta m$). The flat, low-LR tail of the

Step (cumulative, fixed weights)	ΔmAP	$\Delta\Delta m$
soft-NMS σ 0.75→0.5	$+0.16$	$+0.30\pm0.02$
drop RPN objectness	$+0.39$	$+0.75\pm0.25$
RPN top 2000, image-label λ=2	$+0.03$	$+0.14$
average checkpoints e100–180	$+0.73$	$+1.38\pm0.17$
image–box score fusion (class-agn.)	$+0.22$	$+0.41$
min-keep 100	$+0.02$	$+0.04$
lattice snapping	$+0.32$	$+0.62\pm0.21$
total	$+1.87$	$\approx+3.6$

Table 3. Zero-training steps on full-length runs (*vallike*, 3 seeds; final mAP 58.74±0.09).

200-epoch cosine is exactly where averaging helps. Dropping RPN objectness from the score helps because objectness is poorly calibrated for tiny boxes. Snapping is the only data-mining finding that survived testing: $+0.20$ and $+0.36$ mAP on two holdouts (paired bootstrap $P\approx1$).

4.3. The CNN line and full-block adaptation

CNN line (Fig. 3a). YOLO11-L trained on all data reached 37.8 on the board. Faster R-CNN on ConvNeXt-T gave the best macro-F1 of any model at that time (95.3 local) but loose boxes: AP50 was 5 points and AP75 15 points below YOLO11-L. This led to a three-stage cascade and the larger ConvNeXt-S (35.3→37.2). The zero-training steps transferred directly: soft-NMS, score fusion and the box shift gave $+2.8\,\Delta m$ on the board with the same weights (mAP 45.0→46.4). The small-object recipe then added only $+0.3$ (46.5 mAP). DETR-style detectors trailed YOLO11-L on our split (DEIMv2-L -3.4 mAP at 48 epochs, RF-DETR-S ≈ -12), and DINOv3 ViT-B with LoRA behind a plain FPN in the same R-CNN scored only 33.2. The main model differs in its ViTDet neck with normalised per-level convolutions, final-norm tokens and a wider cascade head; we did not isolate which of these mattered. With the main line 11 Δm ahead, we moved this compute to it.

Full blocks *vs*. wide LoRA (Fig. 3b). At 36 epochs, training the last two blocks fully (S3) beat the control by $+2.06$ / $+2.21$ mAP on the two local splits, with gains on harbor, wind turbine and airport. Pretrained or random spatial priors added at most 0.4. A doubled LR diverged. Trained for 150 epochs on all data, however, S3 scored 52.91 mAP on the validation board against 52.66 for the same-length SC2 (+0.25), and averaging epochs 120–140 reached mAP 53.03 / Δm 52.56. That is still below BP7, so the final model kept rank-80 LoRA.

4.4. Data findings and the weak classes

Near-duplicates. About 10% of holdout images have a near-copy in the training set (Fig. 4, right). Local mAP is 83 on these against 40 for images without one, and playground AP is 87 against 12. This inflates local scores and is why we built *vallike* and required a second confirmation.

Bridge and playground. Bridge stays at AP 8 and playground at 33 (Fig. 4, left), although wind turbines, which are smaller, reach 79; size alone is not the cause. Only 53% / 67% of their boxes receive a proposal at IoU 0.5 (96% for wind turbines), and bridge↔playground accounts for 24 of 48 image-classification errors. Evidence from the CNN line points to the labels. A YOLO11-L reaches playground AP50 62.5 on its own training images but 6.4 held out, so it memorises rather than generalises. 33% of bridge and 23% of playground boxes contain no bright SAR return (7% for wind turbines, 0% for ships). Shifting, rescaling or swapping the ground-truth boxes never improved the match (identity was always best), and no image feature separated labelled objects from confident unlabelled detections. Positive-unlabelled loss reweighting (41.28 *vs*. 41.31 mAP), class weights and oversampling all failed to move bridge by more than 0.9 AP; oversampling also cost $-4.2\,\Delta m$ on other classes. We conclude that these labels are only partly recoverable from SAR. This remains a limit of the data, not one we could train away.

4.5. What did not work

Augmentation. 90° rotations cost 5–6 Δm on YOLO models and lost 3.7 mAP over 100 epochs on the cascade; copy-paste and zoom also trailed plain flips. We attribute this to SAR geometry: shadow and layover follow the radar look direction, so rotated images are physically implausible. Oversampling images with small objects helped every short run ($+4$ to $+11$ Δm at 24 epochs), but was not tested at full length. **Other.** EMA, an IoU branch, a 1024-RoI batch, overlapping patches (stride 8, 2.6× compute), halving the LR (-7.2), a LingBot-B backbone, box voting, rank or size priors, and RoI context were all neutral or harmful. RoI context is the most instructive case: $+2.26\,\Delta m$ on one holdout, -0.12 on the other and -0.03 on the board.

4.6. Lessons on evaluation

Three independent cases show local gains shrinking by 5–10× or vanishing on the board. RoI context went from $+2.26\,\Delta m$ to -0.03. The CNN small-object recipe went from $+1.47$ holdout mAP to $+0.17$ mAP on the board. Full-block adaptation went from $+2.1$ mAP at 36 epochs to $+0.25$ at 150. The causes are near-duplicates, single-holdout variance and parts whose advantage disappears as the LR decays. Our rule became: short runs rank ideas, and a change is kept only when a second holdout, a full-length run or the board agrees.

5. Conclusion

A frozen generic backbone (DINOv3), adapted with wide LoRA and paired with a detector rebuilt for sub-patch ob-

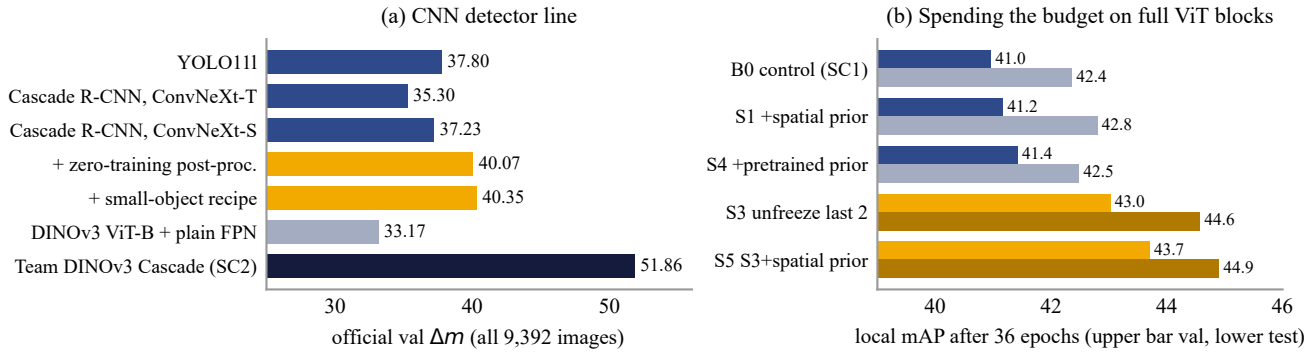


Figure 3. (a) Official validation Δm of the CNN detector line, all trained on the full training set; amber steps reuse the main model's ideas. (b) Screens on the team code at 36 epochs: training the last two ViT blocks fully (amber) leads by $\approx$2 mAP on both local splits, but the lead shrinks to $+0.25$ mAP at 150 epochs on the board (Sec. 4.6).

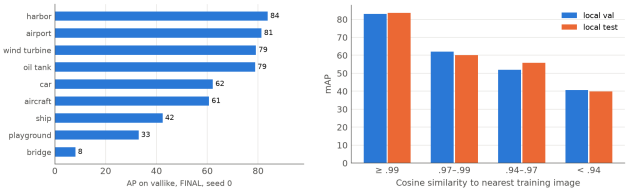


Figure 4. Left: per-class AP of the final recipe on *vallike*. Right: mAP by similarity to the nearest training image (frozen DINOv3 cosine).

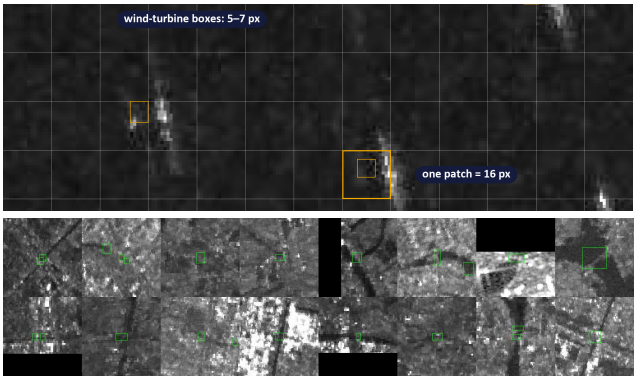


Figure 5. Visual examples (training images, ground truth). Top: wind turbines; the 5–7 px boxes sit beside the bright return and inside one 16 px ViT patch (grid). Bottom: bridge boxes (green), often on dark road crossings with no bright SAR return, which helps explain why bridge AP stays at 8.

jects, carried most of the gain, and patient schedules and careful use of one forward pass supplied the rest. The metric decided where effort went: one mAP point is worth 3.6 F1 points. The remaining gap to the leaders, about 5 mAP, lies at strict IoU on tiny boxes and in two classes whose labels SAR does not fully support. **Limitations.** The submitted model is a single seed; the ablations are 36-epoch runs, and their absolute Δm uses the official reference on a local split. Several Phase 1 choices were made on the validation board, so our validation score is optimistic. Most code was written with AI coding assistants directed by the team.

Compliance. 125,076,960 total / 39,867,360 trainable parameters; one gray 512^2 input, one pass, no TTA or ensembles; pretrained weights DINOv3 LVD-1689M (none trained on SAR); about 514 GPU-hours for the whole project, including tuning.

References

[1] Ahmed Agiza, Marina Neseem, and Sherief Reda. MTLoRA: A low-rank adaptation approach for efficient multi-task learning. In *CVPR*, 2024. 2

[2] Navaneeth Bodla, Bharat Singh, Rama Chellappa, and Larry S. Davis. Soft-NMS – improving object detection with one line of code. In *ICCV*, 2017. 2

[3] Zhaowei Cai and Nuno Vasconcelos. Cascade R-CNN: Delving into high quality object detection. In *CVPR*, 2018. 2

[4] Zhe Chen, Yuchen Duan, Wenhai Wang, Junjun He, Tong Lu, Jifeng Dai, and Yu Qiao. Vision transformer adapter for dense predictions. In *ICLR*, 2023. 2

[5] Alexey Dosovitskiy et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021. 2

[6] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022. 2

[7] Pavel Izmailov, Dmitrii Podoprikhin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization. In *UAI*, 2018. 2

[8] Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-task

learning using uncertainty to weigh losses for scene geometry and semantics. In *CVPR*, 2018. 1

[9] Yanghao Li, Hanzi Mao, Ross Girshick, and Kaiming He. Exploring plain vision transformer backbones for object detection. In *ECCV*, 2022. 2

[10] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *CVPR*, 2017. 2

[11] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A ConvNet for the 2020s. In *CVPR*, 2022. 2

[12] Kevis-Kokitsi Maninis, Ilija Radosavovic, and Iasonas Kokkinos. Attentive single-tasking of multiple tasks. In *CVPR*, 2019. 1

[13] Maxime Oquab et al. DINOv2: Learning robust visual features without supervision. *TMLR*, 2024. 2

[14] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *CVPR*, 2016. 2

[15] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *NeurIPS*, 2015. 2

[16] Oriane Siméoni et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 2

[17] Simon Vandenhende, Stamatios Georgoulis, Wouter Van Gansbeke, Marc Proesmans, Dengxin Dai, and Luc Van Gool. Multi-task learning for dense prediction tasks: A survey. *IEEE TPAMI*, 2021. 1

[18] Jinwang Wang, Chang Xu, Wen Yang, and Lei Yu. A normalized Gaussian Wasserstein distance for tiny object detection. *arXiv preprint arXiv:2110.13389*, 2021. 2

[19] Mitchell Wortsman et al. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *ICML*, 2022. 2

[20] Yian Zhao, Wenyu Lv, Shangliang Xu, Jinman Wei, Guanzhong Wang, Qingqing Dang, Yi Liu, and Jie Chen. DETRs beat YOLOs on real-time object detection. In *CVPR*, 2024. 2